MBU: Scalable and Constant-Round Evaluation of Non-linear Functions in Standard MPC Setting

Min Yang, Dongcan Guo, Zihang Zhou, Jinxuan Du, and Qingshu Meng

Wuhan University, Hubei Province 430072, China qsmeng@126.com

Abstract. After more than four decades of research, multi-party computation (MPC) has achieved remarkable success in handling 2-variable multiplication and comparison-based functions (e.g., ReLU) with practical efficiency. However, for general non-linear functions—such as multiplication of many variables, power, exponential, trigonometric functions, sigmoid, softmax, and GeLU—no native MPC algorithm exists as Beaver-based mutiplication that is constant-round, scalable and exact. Existing solutions rely on either polynomial approximations (trading precision for efficiency), iterative multi-round protocols like Multiplication-to-Addition (M2A) conversion (requiring $\log_2 k$ rounds for k parties), or Function Secret Sharing (FSS) with lookup tables (introducing quantization errors and large storage, mainly limited to 2-4 parties). These approaches suffer from fundamental trade-offs among accuracy, communication rounds, and scalability.

In this work, we propose a unified *mask-broadcast-unmask* design pattern that enables constant-round, scalable and *approximation-free* evaluation of a wide range of non-linear functions. Our contributions include:

- A **general multiplication** protocol for k variables in *one round* with optimal $O(kn)$ communication. When $k = 2$, it reduces to the classic Beaver triple multiplication; when each secret has only one non-zero share and $k = 2$, it becomes the well-known M2A protocol.
- **Power functions** (x^k) in one round.
- **Trigonometric functions** ($\sin x, \cos x$) and **exponential functions** (a^x) in 4 rounds.
- **Sigmoid, softmax** in 6 rounds.

All these protocols are provably secure in the semi-honest model, support arbitrary number of parties, introduce **no approximation error** beyond plaintext floating-point rounding, and require only constant communication rounds (1–6) independent of function complexity. Furthermore, by restricting the random mask to a suitable range (e.g., $0 \leq r < 2^l - 2^{l_x}$), we can reduce the rounds from 1-6 to 1-3. This work fills the long-standing gap for general non-linear functions in standard MPC settings, making privacy-preserving machine learning more practical for modern DNNs.

1 Introduction

Multi-party computation (MPC) has seen tremendous progress over the past four decades, evolving from a theoretical curiosity to a practical tool for privacy-preserving collaborative computation. In particular, MPC has achieved efficient

solutions for two-variable mutliplication and comparison-based functions (e.g., ReLU) using techniques such as Beaver triples, garbled circuits, and secret sharing. However, a fundamental and persistent challenge remains: the secure evaluation of *general non-linear functions*—multiplication of many variables, power, exponential, trigonometric functions, sigmoid, softmax, and GeLU—which are ubiquitous in modern machine learning and statistical analysis.

1.1 The Problem: No Native Algorithm To Non-linear Functions

Despite decades of research, there is no native MPC algorithm for general non-linear functions that simultaneously achieves:

1. **No approximation error** — exact (for algebraic functions) or bitwise equivalent to plaintext floating-point evaluation (for transcendentals);
2. **Constant communication rounds** — independent of function complexity or number of parties;
3. **Scalability** — support large number of participants.

Existing approaches each suffer from critical drawbacks:

- **Polynomial approximations** (e.g., Taylor series, piecewise polynomials) are the most common workaround. They approximate non-linear functions by low-degree polynomials, enabling evaluation via standard MPC multiplication. However, this forces an inherent trade-off: higher-degree polynomials improve accuracy but increase rounds and communication; lower-degree polynomials are efficient but introduce approximation errors that can degrade model accuracy, often requiring fine-tuning. No polynomial approximation can achieve exactness.
- **Multiplication-to-Addition (M2A) conversion** achieves exact results (up to floating-point rounding) by converting multiplicative shares to additive shares. However, existing M2A-based protocols require $\lceil \log_2 k \rceil$ communication rounds for k parties, which becomes a bottleneck in low-latency settings or when many such conversions are needed.
- **Function Secret Sharing (FSS) with lookup tables** precomputes function values for discretized input bins. While achieving constant online rounds, this method introduces **quantization error** due to finite bin granularity. Reducing the error requires exponentially more bins, leading to large storage and offline overhead. Usually they only support 2-4 parties.

In summary, after 40 years of MPC research, there is no native algorithms for general non-linear functions that is constant-round, scalable and exact and avoids the pitfalls of approximation, high rounds, or quantization.

1.2 Our Approach: Mask-Broadcast-Unmask

We introduce a systematic framework based on a simple yet powerful design pattern: *mask-broadcast-unmask*. The idea is to shift complexity to an offline phase that precomputes structured random tuples, and to design an online phase where parties:

1. Mask their shares with precomputed randomness and broadcast the masked values.
2. Reconstruct the masked sum and evaluate the target function (or a simple auxiliary function) locally using standard arithmetic (exact ring/field for algebraic functions, IEEE 754 floating-point for transcendentals).
3. Use precomputed random tuples to remove the effect of randomness, obtaining exact shares of the target function.

This pattern guarantees:

- **Constant rounds** (1–6) — the only communication is the initial broadcast.
- **No approximation error** — transcendentals are evaluated directly on the reconstructed masked value using standard math libraries; only the inherent rounding of floating-point arithmetic remains, identical to plaintext evaluation.
- **Linear communication** — each party broadcasts $O(1)$ values per function.

1.3 Our Contributions

We instantiate this pattern for a comprehensive set of non-linear functions:

- **General multiplication** ($\prod_{j=1}^{k} x_j$): 1 round, $O(kn)$ communication. When $k = 2$, it reduces exactly to the classic Beaver triple multiplication[1]. When each secret has only one non-zero share and $k = 2$, it becomes the well-known M2A protocol[9].
- **Power function** (x^k): 1 round, $O(n)$ communication.
- **Trigonometric functions** ($\sin x, \cos x$): 3 rounds of communication for 2 participants; 4 round for $n > 2$ participants.
- **Exponential functions** (a^x): 3 rounds of communication for 2 participants; 4 round for $n > 2$ participants.
- **Sigmoid, softmax**:6 rounds, built by composing the exponential and multiplication protocols.

All protocols are provably secure in the semi-honest model with up to $n - 1$ corruptions, and are fully scalable (any number of parties). The offline phase only requires precomputed random tuples, which can be generated by existing techniques (PCG, OT extension, or even a trusted dealer).

1.4 Comparison with Prior Art

Table 1 contrasts our work with existing approaches. To the best of our knowledge, no prior work simultaneously provides:

- Exact computation (no polynomial or quantization error);
- Constant online rounds (1–6);
- Linear online communication;
- Coverage of all common non-linear functions (multiplication, power, exp, trig, sigmoid, softmax, GeLU).

1.5 Paper Organization

The rest of the paper is organized as follows. Section 2 reviews preliminaries. Section 3 presents our protocols for general multiplication, power, exponential, trigonometric, sigmoid, softmax, and GeLU. Section 4 discusses offline precomputation. Section 5 provides security analysis. Section 6 discusses related work, and Section 7 concludes.

2 Preliminaries

2.1 System Model

We consider n parties $P_1, \ldots, P_n$ holding additive secret shares of a value $x \in \mathbb{Z}_{2^l}$ (or a finite field), such that $x = \sum_{i=1}^{n} x_i$. The goal is to securely compute a function $f(x)$, with each party obtaining an additive share of $f(x)$. We assume a semi-honest adversary controlling up to $n-1$ parties. All protocols are presented in the standard offline/online model, where an offline phase precomputes correlated randomness independent of the inputs.

2.2 Additively Secret Sharing (ASS)

ASS is a core MPC primitive for secure computation, offering information-theoretic security against semi-honest adversaries. It is widely used in PPML systems [19,22,18] due to its simplicity and efficiency for linear operations. We adopt the ASS-based MPC for the evaluation of non-linear functions.

Sharing: To share $x \in \mathbb{Z}_{2^l}$, each P_i generates a random $\langle x \rangle_i \in \mathbb{Z}_{2^l}$ such that $x = \sum_{i=1}^{n} \langle x \rangle_i \mod 2^l$.

Reconstruction: Each P_i broadcasts $\langle x \rangle_i$, and locally compute $x = \sum_{i=1}^{n} \langle x \rangle_i \mod 2^l$.

Addition: For shared values x and y, each party computes local sums: $\langle x + y \rangle_i = \langle x \rangle_i + \langle y \rangle_i \mod 2^l$.

Public Scalar Multiplication: For $a \in \mathbb{Z}_{2^l}$ (public) and $\langle x \rangle$, each party multiplies locally: $\langle ax \rangle_i = a \cdot \langle x \rangle_i \mod 2^l$.

2.3 Computational Model

Our protocols operate in two distinct computational settings depending on the function type.

Algebraic Functions. For functions such as multiplication, power, computation is performed over a ring $\mathbb{Z}_{2^l}$. All operations—addition, multiplication, and reconstruction—are exact and deterministic. The protocols produce shares that reconstruct to the exact value(only with random rounding error) that would be obtained by evaluating the function in the same algebraic structure.

Transcendental Functions. For functions such as exp, sin, cos, sigmoid, softmax, and GeLU, each input is masked $x_i + r_i \mod 2^l$. The HP reconstructs

a masked value $x + r \mod 2^l$ first and then turns it into a floating-point number. The evaluation of function is performed over floating-point numbers using standard arithmetic (e.g., IEEE 754). We assume that all parties have access to a deterministic floating-point implementation of these functions (e.g., from a standard math library). The final reconstructed output is identical to the result of evaluating the function on the plaintext input (only with random rounding error)using the same floating-point arithmetic. No additional approximation is introduced by the MPC protocol.

3 Protocols for Non-linear Functions

3.1 Core Design Pattern

All our online protocols follow a simple, unified pattern that guarantees constant rounds:

1. Each party P_i masks its share x_i with a precomputed random r_i and broadcasts $x_i + r_i$.
2. All parties reconstruct $x + r = \sum_i (x_i + r_i)$ and compute $g(x + r)$ locally, where g is an auxiliary function (often the same as f or a simple variant).
3. Using precomputed shares of functions $h(r)$, where $h()$ is a function related to $f()$, each party removes the effect of r to obtain a share of $f(x)$.

This structure guarantees constant rounds because the only communication is the initial broadcast; all subsequent steps are local.

We now present concrete instantiations of this pattern for the set of common non-linear functions used in privacy-preserving machine learning. All protocols are exact (as defined above), require constant rounds, and have total communication linear in n.

3.2 Multi-Variable Multiplication

Offline Precomputation: Output ASS shares $\langle R_S \rangle_i$ of R_S for every non-empty subset $S \subseteq \{1, \ldots, k\}$, where $R_S = \prod_{j \in S} r_j$, and $r_1, r_2, \ldots, r_k \in \mathbb{Z}_{2^l}$ are k random numbers. The empty set $S = \emptyset$ gives $R_\emptyset = 1$, no shares needed.
Online Phase (1 round)
Input: Each P_i holds $\langle x_1 \rangle_i, \ldots, \langle x_k \rangle_i$ with $x_j = \sum_i \langle x_j \rangle_i$.
Output: Each P_i obtains $\langle \prod_{j=1}^k x_j \rangle_i$.

1. Each party P_i (in parallel) compute $a_{j,i} = \langle x_j \rangle_i + \langle r_j \rangle_i$ for $j = 1, \ldots, k$. Broadcast all k values $a_{1,i}, \ldots, a_{k,i}$ to all other parties.
2. Each party P_i reconstructs $A_j = \sum_{m=1}^n a_{j,m} = x_j + r_j$ (public), $j = 1, \ldots, k$.
3. Each party P_i (in parallel) set $\langle V \rangle_i = 0$. For each non-empty $S \subseteq \{1, \ldots, k\}$, compute $T_{S,i} = (-1)^{|S|} \left(\prod_{j \notin S} A_j \right) \cdot \langle R_S \rangle_i$, and then $\langle V \rangle_i \leftarrow \langle V \rangle_i + T_{S,i}$.
4. Designate P_1 to compute $P = \prod_{j=1}^k A_j$ (public). P_1 sets $\langle \prod_{j=1}^k x_j \rangle_1 = \langle V \rangle_1 + P$. For $i = 2, \ldots, n$, P_i sets $\langle \prod_{j=1}^k x_j \rangle_i = \langle V \rangle_i$.

Correctness: By inclusion–exclusion,

$$\prod_{j=1}^{k} x_j = \sum_{S\subseteq\{1,\ldots,k\}} (-1)^{|S|}\Big(\prod_{j\notin S}(x_j+r_j)\Big)\Big(\prod_{j\in S} r_j\Big) = P + \sum_{S\neq\emptyset}(-1)^{|S|}\Big(\prod_{j\notin S} A_j\Big)R_S.$$

Thus the sum of the output shares equals $\prod_{j=1}^{k} x_j$.

Communication: 1 round, $O(kn)$ elements. Without this protocol, to compute the product of k variables, it takes $log_2 k$ rounds of communication with classic Beaver-based multiplication.

Precomputation: $2^k - 1$ random product shares (only practical for small k, say $k \leq 5$). For large k, say k=64, it is not practical to use our scheme directly, a hybrid scheme can be used: the 64 variables are divided into 16 groups, with each group having 4 variables. Each group can be solved by the multi-variable multiplication protcol. The whole comunication rounds is reduced from 6 to 3.

Three special cases:

Case 1: The classic Beaver-based multiplication. The multi-variable multiplication protocol with k=2 becomes the classic Beaver-based Multiplication[1], which is the ruling method of multiplication in MPC.

1. Each party P_i (in parallel) compute $\langle x_1\rangle_i + \langle r_1\rangle_i$, $\langle x_2\rangle_i + \langle r_2\rangle_i$. Broadcast them to all other parties.
2. All parties reconstruct $e = \sum_{i=1}^{n}(< x_1 >_i + < r_1 >_i) = x_1 + r_1$ and $f = \sum_{i=1}^{n}(< x_2 >_i + < r_2 >_i) = x_2 + r_2$.
3. P_1 computes $ef + e < r_2 >_1 + f < r_1 >_1 + < r_1 r_2 >_1$; other party P_i computes $e < r_2 >_i + f < r_1 >_i + < r_1 r_2 >_i$.

Case 2: the M2A protocol. If each variable x_i is specially shared with only one share is nonzero, then the protocol becomes the M2A protocol proposed in the paper[9] and used in papers[6,27]. Theoretically, with our scheme the communication rounds is reduced from $log_2 k$ to one round. In practice, communication rounds is reduced from $log_2 k$ rounds to $log_a k$ rounds, where $a = 3, 4 or 5$.

Case 3: The power function. When all $x_i, i = 1, 2, \cdots, k$ are equal to each other, the protocol becomes the power function. Considering the simplified steps and its important usage, we discuss this case in the next subsection.

3.3 Power Function (x^k)

For $k \geq 2$, we compute x^k exactly in one round.

Offline Precomputation: Output ASS shares $\langle(-r)^{k-t}\rangle_i, i = 1, 2, \cdots, n$ of $(-r)^{k-t}$ for $t = 0, 1, \ldots, k-1$. Distribute $\langle(-r)^{k-t}\rangle_i, i = 1, 2, \cdots, n, t = 0, 1, \ldots, k-1$ to each party P_i and the ASS shares $\langle r\rangle_i, i = 1, 2, \cdots, n$ of r.

Online Phase:

Input: Each P_i holds ASS shares $\langle x\rangle_i$ such that $x = \sum_{i=1}^{n}\langle x\rangle_i$.

Output: Each P_i obtains the ASS share $< x^k >_i$ of x^k.

1. Each party P_i broadcasts $\langle x\rangle_i + \langle r\rangle_i$.
2. Each party P_i reconstructs $x+r = \sum_{j=1}^{n}(\langle x\rangle_j + \langle r\rangle_j)$, and locally computes the public powers $(x+r)^t$ for $t = 0, 1, \ldots, k$.
3. Each party computes its share of x^k as:

$$\langle x^k\rangle_i = \sum_{t=0}^{k-1} \binom{k}{t} (x+r)^t \cdot \langle (-r)^{k-t}\rangle_i.$$

Let $tmp = \langle x^k\rangle_1$. P_1 computes its shares of x^k as $\langle x^k\rangle_1 = tmp + (x+r)^k$.

Correctness: By the binomial theorem,

$$x^k = (x+r-r)^k = \sum_{t=0}^{k} \binom{k}{t} (x+r)^t \cdot (-r)^{k-t} = \sum_{t=0}^{k-1} \binom{k}{t} (x+r)^t \cdot (-r)^{k-t} + (x+r)^k.$$

Since each party P_i holds $\langle (-r)^{k-t}\rangle_i$, the local linear combination yields a correct share of x^k. "1" is unnecessary to secretly shared.

Communication: 1 round, $O(n)$ elements. Without this protocol, to compute the power x^k with Beaver-based mulitplicaiton, it take $log_2 k$ rounds. This is vey important for evaluation of nonlinear functions, such as division, log(x) and nth root extraction functions. As there is no similar additive mask solution to them, polynomial approximatoin seems to be a possible solution, which we will discuss in another paper for space limitation.

3.4 Wrap Protocol

The wrap protocol determines whether $x+r$ exceeds 2^l. It returns 1 if $x+r \geq 2^l$, and 0 otherwise. Here $x = \sum_{i=1}^{n} x_i \mod M$ and $r = \sum_{i=1}^{n} r_i \mod M$ are shared among n parties, with each party P_i holding x_i and r_i, and $M = 2^l$.

The protocol is based on the identity:

$$(x_i + r_i) = (x_i \bmod M) + M \left\lfloor \frac{x_i}{M} \right\rfloor + (r_i \bmod M) + M \left\lfloor \frac{r_i}{M} \right\rfloor,$$

which leads to

$$\left\lfloor \frac{\sum_i (x_i + r_i)}{M} \right\rfloor = \left\lfloor \frac{x+r}{M} \right\rfloor + \left\lfloor \frac{\sum_i x_i}{M} \right\rfloor + \left\lfloor \frac{\sum_i r_i}{M} \right\rfloor.$$

The term $\left\lfloor \frac{\sum_i r_i}{M} \right\rfloor$ can be precomputed offline. Each party can compute $\left\lfloor \frac{\sum_i (x_i+r_i)}{M} \right\rfloor$ from the received broadcasts. If $\left\lfloor \frac{\sum_i x_i}{M} \right\rfloor$ can be obtained, then $\lfloor (x+r)/M \rfloor$ (i.e., the wrap value) follows. The sign protocol[17] is listed in Appendix A.

Offline Phase: Precompute $r_i \in Z_M$ and send to each party P_i. Precompute $\left\lfloor \frac{\sum_i r_i}{M} \right\rfloor$ and broadcast to each party.

Online Phase:

1. On the larger ring $\mathbb{Z}_{M^2}$, the parties run the sign protocol with inputs $\{x_1 - j2^l + 1, x_2, \ldots, x_n\}$ to decide whether $x > j2^l$ for $j = 0, 1, \cdots, n-1$. The largest j for which the inequality holds gives $\left\lfloor \frac{\sum_i x_i}{M} \right\rfloor$.
2. Each party computes $x_i + r_i$ (without modulo) and broadcasts it.
3. All parties compute $\left\lfloor \frac{\sum_i (x_i + r_i)}{M} \right\rfloor$ over a larger ring (e.g., $\mathbb{Z}_{M^2}$) such that $\sum_i (x_i + r_i)$ is within the modulus without overflow. Then using the value of $\left\lfloor \frac{\sum_i x_i}{M} \right\rfloor$ from step 1, each party computes $\lfloor (x+r)/M \rfloor = \left\lfloor \frac{\sum_i (x_i + r_i)}{M} \right\rfloor - \left\lfloor \frac{\sum_i x_i}{M} \right\rfloor - \left\lfloor \frac{\sum_i r_i}{M} \right\rfloor$ as the wrap value $\mathrm{wrap}(x, r, 2^l)$.

The output j does not leak information about x in the sense of $x \bmod M$. As step 2 and 3 are independent from step 1, they can run in parallelly. The protocol requires 4 rounds of communication.

Practical optimization: The wrap protocol requires expensive 4 rounds of communication. Though in the special case of two parties, the wrap protocol can be executed in 3 rounds, it is still expensive. To make our schemes more practical while preserving strong security, we observe that in typical machine learning settings the effective bit-length l_x of the input x is small (e.g., $l_x \leq 14$), whereas the ring modulus is 2^l with $l = 32$ or 64. The expensive wrap protocol (4 rounds) is only needed when $x + r \geq 2^l$. We can entirely avoid this overhead by choosing the random mask r such that $x + r < 2^l$ always holds. Two simple approaches achieve this:

1. **Probabilistic security:** let r be uniform over $\mathbb{Z}_{2^l}$. The overflow probability is at most $2^{l_x}/2^l = 2^{l_x - l}$, which for $l_x = 14$, $l = 64$ is only 2^{-50} — negligible in practice.
2. **Deterministic security:** restrict r to the range $0 \leq r < 2^l - 2^{l_x}$. Since $x < 2^{l_x}$, we have $x + r < 2^{l_x} + (2^l - 2^{l_x}) = 2^l$. This eliminates only a negligible fraction $2^{l_x}/2^l = 2^{l_x - l}$ of the ring, thus preserving almost full entropy.

Consequently, the wrap protocol is omitted entirely, saving 4 communication rounds for all protocols that rely on it (exponential, trigonometric functions, etc.). However, in the following protocol, we still use the wrap protocol for completeness.

3.5 Trigonometric Functions ($\sin x$, $\cos x$)

- **Offline Precomputation:** Output shares $< r >_i$ of random number $r \in Z_{2^l}$ and shares $< sin(r) >_i, < cos(r) >_i$ of $\sin(r)$ and $\cos(r)$ respectively , where r is turne into real number in accordin to the precison of x(see Section 4).
- **Online Phase:**
  Input: Each P_i holds $\langle x \rangle_i$ such that $x = \sum_i \langle x \rangle_i$.
  Output: Each P_i obtains $\langle sin(x) \rangle_i$ and/or $\langle cos(x) \rangle_i$.

1. Each party broadcasts $\langle x\rangle_i + \langle r\rangle_i$, and reconstruct $x + r = \sum_{j=1}^{n}(\langle x\rangle_j + \langle r\rangle_j) \mod 2^l$.
2. Each party turns $x + r$ into real number in according to the precision of x and locally computes $\sin(x + r)$ and $\cos(x + r)$ using a standard math library.
3. Each party runs the wrap protocol to obtain $w = wrap(x, r, 2^l)$.
4. Each party computes

   $< sin(x) >_i = sin(x+r+w2^l) \cdot < cos(r) >_i - \cos(x+r+w2^l) \cdot < sin(r) >_i$,

   $< cos(x) >_i = cos(x+r+w2^l) \cdot < cos(r) >_i + \sin(x+r+w2^l) \cdot < sin(r) >_i$,

   where $sin(x + r + w2^l) = sin(x + r)cos(w2^l) + cos(x + r)sin(w2^l)$, and $cos(c + r + w2^l) = cos(x + r)cos(w2^l) - sin(x + r)sin(w2^l)$.

As step 1 and 2 are independent from step 3, they can run in parallely. The protocol requires 4 rounds of communication.

3.6 Exponential (a^x)

- **Offline Precomputation:** Output the shares $< r >_i$ of random r and shares $< a^{-r} >_i$ of a^{-r}, $i = 1, 2, \cdots, n()$.
- **Online Phase:**
  Input: Each P_i holds $\langle x\rangle_i$ such that $x = \sum_i \langle x\rangle_i$.
  Output: Each P_i obtains ASS share $\langle a^x\rangle_i$ of a^x.
  1. Each party broadcasts $\langle x\rangle_i + \langle r\rangle_i \mod 2^l$, and locally reconstruct $x + r = \sum_{j=1}^{n}(\langle x\rangle_j + \langle r\rangle_j) \mod 2^l$.
  2. Each party run the wrap protocol to calculate $w = wrap(x, r, 2^l)$.
  3. Each party turns $x + r + w2^l$ into real numer in according to the precision of x and computes a^{x+r+w2^l} locally.
  4. Each party outputs $a^{x+r+w2^l} \cdot < a^{-r} >_i$.

As step 1 and 2 are independent from step 3, they can run in parallely. The protocol requires 4 rounds of communication.

3.7 Sigmoid

The sigmoid function is defined as:

$$\text{sigmoid}(x) = \frac{e^x}{1 + e^x}.$$

The sigmoid protocol is as follows:
Offline precomputation: Output the ASS shares $\langle e^s\rangle$ of e^s, with s a random number.
Online pahse:
Input: Each P_i holds $\langle x\rangle_i$ such that $x = \sum_i \langle x\rangle_i$.
Output: Each participant P_i obtains an ASS share of sigmoid(x).

Protocol Steps:

1. Run the exponential protocol to obtain shares $\langle e^x \rangle_i$.
2. Each participant takes $\langle e^x + 1 \rangle_i$ and $\langle e^s \rangle_i$ as inputs, runs the multiplication protocol to obtain $\langle (e^s e^x + e^s) \rangle_i$, and broadcasts it; each party takes $\langle e^x \rangle_i$ and $\langle e^s \rangle_i$ as inputs, runs the multiplication protocol to obtain $\langle (e^s e^x) \rangle_i$.
3. Each participant P_i aggregates the received shares to obtain $t = e^s e^x + e^s$, and locally computes $\langle e^{x+s} \rangle_i / t$ as the share of sigmoid(x).

The protocol requires 6 rounds of communication, and offers provable security. Since tanh can be linearly defined from sigmoid, the Tanh function can be implemented analogously using this protocol and is not elaborated further.

3.8 Softmax

The softmax function is defined as:

$$\text{softmax}(x_m) = \frac{e^{x_m}}{\sum_{j=1}^{k} e^{x_j}}.$$

offline precomputation: Output ASS shares $\langle e^r \rangle$ of e^r, with r a random number.
Online pahse:
Input: Each P_i holds $\langle x_j \rangle_i$, with $x_j = \sum_i \langle x_j \rangle_i$, $j = 1, 2, \cdots, k$.
Output: Each participant P_i obtains an additive secret share of softmax(x_m).
Protocol Steps:

1. Each party runs the exponential protocol to obtain ASS shares of all e^{x_j}, $j = 1, 2, \ldots, k$. Each participant P_i computes the sum of its own shares:

$$s_i = \left\langle \sum_{j=1}^{k} e^{x_j} \right\rangle_i.$$

2. Each party takes s_i and $\langle e^r \rangle_i$ as inputs, runs the multiplication protocol to obtain $< \sum_{j=1}^{k} e^{x_j + r} >_i$, and broadcasts the result; each party takes $\langle e^{x_m} \rangle_i$ and $\langle e^r \rangle_i$ as inputs, runs the multiplication protocol to obtain $\langle e^{x_m + r} \rangle_i$.
3. Each participant aggregates the received shares to obtain

$$S = \sum_{j=1}^{k} e^{x_j + r},$$

and locally computes $\langle e^{x_m + r} \rangle_i / S$ as the share of softmax(x_m).

The protocol requires 6 rounds of communication and offers provable security. In step 1, if each party broadcast s_i to reconstruct the denominator, the denominator will leak some information (the average value of each variable, for example) though it is impossible to infer exact value of each x_i for $i = 1, 2. \cdots, k$.

Our MBU scheme also supports GeLU. However, as the protocol is based on the two approximation represenatations[12], it is not an exact protocol. We list it in Appendix B.

4 Offline Precomputation

Our protocols require two types of precomputed tuples:

Type I (Algebraic Correlations). For multiplication and power functions, we require Beaver triples $(r, s, r \cdot s)$ and their generalizations. These can be generated efficiently using pseudo-random correlation generators (PCG) [4,15] or oblivious transfer extensions [13]. These techniques are well-established and can be instantiated with linear communication and constant rounds in the offline phase.

Type II (Transcendental Correlations). For transcendental functions, we require tuples $(r, h(r))$ where $h \in \{\sin, \cos, \exp, \text{etc.}\}$. Unlike algebraic correlations, these involve non-linear, non-polynomial relations and cannot be generated by PCG-based methods by now.

Generating Transcendental Correlations. The generation of such tuples is an **independent research problem** that has received increasing attention. Several approaches exist:

- **Trusted Dealer Model.** A common simplification in MPC literature [2] assumes a trusted offline dealer that generates random r, computes $h(r)$ locally, and distributes secret shares. This suffices for security proofs when the dealer does not interact with online inputs.
- **Function Secret Sharing (FSS).** Recent work [3] extends FSS[5] to support trigonometric functions via piecewise polynomial approximation [25]. For exponential functions, FSS-based constructions remain an open challenge, though fixed-point approximations can be used [23,21].
- **Secure Computation in the Offline Phase.** The parties can jointly compute $h(r)$ from shares of r using any existing MPC protocol (e.g., piecewise and high-degree polynomial approximation, garbled circuits or SPDZ) in the offline phase. Since the offline phase is input-independent and can tolerate higher latency, this approach is feasible in principle, though it may incur higher computational overhead.

Scope of This Work. The focus of this paper is the **online phase**: we provide constant-round, low-communication protocols for non-linear functions assuming the existence of the required precomputed tuples. The offline generation of transcendental tuples is an important orthogonal direction. We note that **our protocols do not require any new offline techniques**—any secure method for generating shares of $h(r)$ suffices. The development of more efficient offline generation for these tuples is left as future work.

5 Security Analysis

All protocols are secure in the semi-honest model with up to $n - 1$ corrupted parties. We prove security via a standard simulation argument.

5.1 Ideal Functionality

For each function f, we define an ideal functionality $\mathcal{F}_f$ that receives inputs $x_1, \ldots, x_n$ from parties, computes $f(x)$ (over the appropriate domain: ring/field for algebraic functions, floating-point for transcendental), and outputs shares to all parties.

5.2 Simulator Construction

Consider the online phase of a protocol for f. The only values revealed to parties are the masked shares $x_i + r_i$ (broadcast in round 1). Since each r_i is uniformly random and independent of the input, the masked value $x + r$ is uniformly distributed. A simulator can generate a random value u and set the simulated broadcast to u.

For transcendental functions, each party broadcasts $x_i + r_i$ leaks nothing because r_i is chosen uniformly in the ring. The sign's security lies in the two mask techniques: addtive mask in additve group and the multiplicative mask in a multiplicative group, which leaks nothing too. The broadcast in wrap protocol $x_i + r_i$ leaks nothing too.

5.3 Hybrid Argument

We prove security by composing the offline generation of precomputed tuples (which is input-independent and secure by construction of the underlying primitive) with the online protocol. The online protocol only uses the precomputed shares in linear combinations or as input to secure multiplcaion protocol, preserving the secret-sharing property. Formal security proofs for each protocol follow by constructing a simulator that replaces the precomputed values with random values consistent with the revealed masked values. For functions (multiple variable multiplication, power function) with 1 round communication, their proof is similar to the proof of classic Beaver-based multiplication. For exponential function, Trigonometric Functions, sigmoid, softmax and GeLU, by UC principle, as each step is secure, the whole protocol is secure.

Due to space constraints, we only provide the full simulation for sigmoid and softmax.

Theorem 1 (Sigmoid). *Let $\pi_{sigmoid}$ be the protocol described in Algorithm* **??** *(or Algorithm 5). $\pi_{sigmoid}$ securely computes* $\mathrm{sigmoid}(x)$ *in the presence of a semi-honest adversary that corrupts any $t < n$ parties (including the worst case $t = n - 1$).*

Proof. The protocol consists of one exponential protocol, two multiplication protocols, and one broadcast of $\langle e^{s+x} + e^s \rangle_i$. The adversary's view consists of:

- the view of the exponential protocol (broadcast values $\langle x \rangle_i + \langle r \rangle_i$ and public $x + r$);

- the views of the two multiplication protocols(broadcast values of form $\langle e^x \rangle_i + \langle \alpha \rangle_i$, $\langle e^s \rangle_i + \langle \beta \rangle_i$, and public values of the form $e^x + \alpha$, $e^s + \beta$ with random α, β;
- the broadcast shares $\langle e^{s+x} + e^s \rangle_i$ and their sum $e^{s+x} + e^s$.

We construct a simulator $\mathcal{S}$ as follows. $\mathcal{S}$ knows the corrupted parties' inputs $\langle x \rangle_i$ $(i \in \mathcal{C})$ and receives from the ideal functionality the shares $\langle \text{sigmoid}(x) \rangle_i$ for $i \in \mathcal{C}$. $\mathcal{S}$ randomly generates all precomputed random tuples (e.g., r, s, and Beaver triples) independently of the inputs, since the offline phase is input-independent.

For each broadcast value (e.g., $\langle x \rangle_i + \langle r \rangle_i$,$\langle e^x \rangle_i + \langle \alpha \rangle_i$, etc.), $\mathcal{S}$ computes the exact values for corrupted parties (using their known inputs and precomputed randomness) and chooses uniformly random values for honest parties such that the total sum matches the public value. The public value $x + r$ is uniformly random because r is random; similarly, each masked value in the multiplications is uniform.

For the final denominator $e^{s+x} + e^s$, note that its distribution is independent of x: since s is uniform, $e^{s+x} + e^s = e^s(e^x + 1)$ has the same distribution for any x. Hence $\mathcal{S}$ can sample a random s' and set the denominator to $c \cdot e^{s'}$ for an arbitrary constant c (e.g., $c = 2$, corresponding to $x = 0$) without knowing x. because at least one honest party's share is unknown to the adversary, the joint distribution is uniform and indistinguishable from the real execution.

All messages received by the adversary are either exact values derived from its known inputs (for corrupted parties) or uniformly random values (for honest parties). Moreover, every public value either is uniformly random (for sums involving independent masks) or has a distribution independent of the input x (for the final denominator), and there is always at least one unknown share from an honest party. Therefore, the simulated view is statistically indistinguishable from the real view.

Theorem 2 (Softmax). *Let $\pi_{softmax}$ be the protocol described in Algorithm 6. $\pi_{softmax}$ securely computes* $\text{softmax}(x_m)$ *in the presence of a semi-honest adversary that corrupts any $t < n$ parties (including the worst case $t = n - 1$).*

Proof. The protocol consists of three phases:

- k parallel exponential protocols (Algorithm 4) to obtain $\langle e^{x_j} \rangle_i$ for $j = 1, \ldots, k$;
- two parallel Beaver multiplications:
  - one to compute $\langle \sum_{j=1}^{k} e^{x_j + r} \rangle_i$ from $\langle e^r \rangle_i$ and $\langle \sum_{j=1}^{k} e^{x_j} \rangle_i$ (where the latter is obtained by local addition of $\langle e^{x_j} \rangle_i$);
  - another to compute $\langle e^{x_m + r} \rangle_i$ from $\langle e^{x_m} \rangle_i$ and $\langle e^r \rangle_i$;
- a broadcast of the shares $\langle \sum_{j=1}^{k} e^{x_j + r} \rangle_i$ to reveal the public denominator $S = \sum_{j=1}^{k} e^{x_j + r}$.

The adversary's view consists of the views of these sub-protocols and the final broadcast.

We construct a simulator $\mathcal{S}$ as follows. $\mathcal{S}$ knows the corrupted parties' inputs $x_{j,i}$ for $j = 1, \ldots, k$, $i \in \mathcal{C}$ (where $\mathcal{C}$ is the set of corrupted parties, $|\mathcal{C}| \leq n-1$), and receives from the ideal functionality the shares $\langle \text{softmax}(x_m) \rangle_i$ for $i \in \mathcal{C}$. $\mathcal{S}$ randomly generates all precomputed random tuples (e.g., r and Beaver triples) independently of the inputs, since the offline phase is input-independent.

Simulation of the exponential protocols. For each j, the exponential protocol reveals broadcast values $x_{j,i} + r_{j,i}$ (where $r_{j,i}$ are shares of a random r_j) and the public value $x_j + r_j$. For corrupted parties $i \in \mathcal{C}$, $\mathcal{S}$ computes the exact broadcast values using the known $x_{j,i}$ and the precomputed $\langle r_j \rangle_i$. For honest parties, $\mathcal{S}$ chooses uniformly random values such that the sum of all broadcasts equals a uniformly random public value (since r_j is random). The resulting view is indistinguishable from the real one, as argued in the analysis of 1-round protocols.

Simulation of the two parallel Beaver multiplications. The two multiplications are independent and can be simulated in parallel. For each multiplication, the protocol reveals masked values of the form $A + \alpha$ and $B + \beta$ with random α, β, where (A, B) are the two inputs being multiplied.

For the denominator multiplication: - Inputs are $A = e^r$ and $B = \sum_{j=1}^{k} e^{x_j}$. - $\mathcal{S}$ knows the corrupted parties' shares of A and B (from precomputation and from summing the exponential outputs). It can compute their exact masked values. - For honest parties, $\mathcal{S}$ chooses random masks such that the public sums are uniformly random (because α, β are random).

For the numerator multiplication: - Inputs are $A' = e^r$ and $B' = e^{x_m}$. - Similarly, $\mathcal{S}$ knows the corrupted parties' shares and simulates honest parties' broadcasts with random values.

Thus the simulated multiplication views are indistinguishable from the real ones.

Simulation of the broadcast. In the real protocol, each party broadcasts its share $\langle S \rangle_i = \langle \sum_{j=1}^{k} e^{x_j + r} \rangle_i$ (from the denominator multiplication). The sum of these shares is $S = \sum_{j=1}^{k} e^{x_j + r}$. We claim that the distribution of S is **independent of the input vector** $(x_1, \ldots, x_k)$. Indeed, $S = e^r \cdot \sum_{j=1}^{k} e^{x_j}$. Since r is uniformly random and independent of all x_j, the factor e^r is a random positive scaling. For any two input vectors $\mathbf{x}$ and $\mathbf{x}'$, the distributions of S are identical (both are e^r times a constant). Therefore, $\mathcal{S}$ can simulate S without knowing the inputs: it picks a random r' and sets $S = c \cdot e^{r'}$ for an arbitrary constant c (e.g., $c = k$, corresponding to all $x_j = 0$). $\mathcal{S}$ then generates shares $\langle S \rangle_i$ for corrupted parties arbitrarily (e.g., uniformly random) and sets the honest parties' shares such that the sum equals S. Because at least one honest party's share is unknown to the adversary, the joint distribution of the shares is uniform and indistinguishable from the real execution.

Simulation of the final division. After the broadcast, each party locally computes $\langle \text{softmax}(x_m) \rangle_i = \langle e^{x_m + r} \rangle_i / S$. $\mathcal{S}$ already knows the ideal output shares $\langle \text{softmax}(x_m) \rangle_i$ for corrupted parties. It simply outputs these values as the result of the division step. (Correctness of the protocol ensures that the division yields exactly these shares.)

Indistinguishability argument. All messages received by the adversary are either:

- exact values computed from its known inputs (for corrupted parties), or
- uniformly random values (for honest parties' broadcasts) with the same distribution as in the real protocol.

Every public sum (e.g., $x_j + r_j$, masked multiplication values, and the final denominator S) is either uniformly random or has a distribution independent of the inputs. Since at least one honest party's share remains unknown to the adversary, the joint distribution of the adversary's view is statistically identical to that in the real execution. Hence the simulated view is statistically indistinguishable from the real view.

6 Related Work

We categorize existing secure inference schemes for Transformers according to how they handle non-linear functions (Softmax, GeLU, SiLU, etc.). Table 1 summarizes the four classes.

6.1 Polynomial Approximation: Trading Accuracy for Efficiency

The most common approach approximates complex non-linear functions using low-degree polynomials, piecewise polynomials, or Taylor expansions. This reduces communication and rounds but inevitably introduces approximation errors that may affect model accuracy.

Hybrid HE/MPC frameworks. BOLT [20] approximates GeLU with a degree-4 polynomial and Softmax with a clipped Taylor series, achieving near-floating-point accuracy without fine-tuning. BumbleBee [16] uses a 4-segment polynomial (degree-3 and degree-6) for GeLU and the approximation $(1+x/2^n)^{2^n}$ for Softmax, reducing communication by 80%–90% but incurring up to 11 ULP error for GeLU. BLB [26] inherits BumbleBee's piecewise approximations.

Three-party frameworks. PUMA [8], Ditto [24], Iron [11], and Nimbus [28] all rely on polynomial or piecewise approximations. MPCFormer [14] replaces GeLU with a quadratic and Softmax with a division of quadratics, causing about 5% accuracy drop that requires fine-tuning.

Limitation. All these methods face an inherent trade-off: higher-degree polynomials improve accuracy but increase communication rounds and computation; lower-degree polynomials are more efficient but risk accuracy degradation. None achieves exact computation without approximation.

6.2 Multiplication-to-Addition (M2A) Based Exact Computation

A different line of work computes non-linear functions exactly (up to floating-point rounding) by converting multiplicative secret shares to additive shares using interactive protocols.

SecureTLM [6] uses hybrid secret sharing (additive and multiplicative). To compute Softmax, each party locally computes $e^{[x_m]_i} \cdot \langle\gamma\rangle_i$ and then invokes a conversion protocol F_{trans} to obtain additive shares. This yields exact results (no polynomial or LUT approximation), but the conversion requires $\lceil \log_2 N \rceil$ communication rounds for N parties.

Limitation. While M2A-based schemes avoid polynomial approximation error, their logarithmic round complexity makes them less suitable for low-latency applications.

6.3 FSS-Based Lookup Tables: Quantization Error

Function secret sharing (FSS) enables low online communication by precomputing keys for lookup tables (LUTs). These schemes discretize the input domain into a finite number of bins and precompute function values for each bin.

SIGMA [10] builds FSS-based protocols for GeLU, Softmax, and layer normalization, achieving GPU-accelerated inference. Its GeLU uses an LUT for $\delta(x) = \text{ReLU}(x) - \text{GeLU}(x)$ on $[-4, 4]$ with 256 entries (6-bit precision), which introduces quantization error. The output differs from plaintext floating-point computation due to discretization.

Mosformer [7] extends FSS to maliciously secure three-party computation using verifiable DPFs (VDPFs). Its GeLU also relies on a precomputed LUT of 256 entries, incurring similar quantization error.

Limitation. Finite LUT size inevitably causes approximation (quantization error). Finer granularity reduces error but increases key size and offline overhead. Thus, these schemes trade off precision against storage and preprocessing cost.

6.4 Our Scheme: Only Random Truncation Error

In contrast to the three categories above, our framework achieves the following:

- **Algebraic functions** (multiplication, power): exact over the underlying ring/field, identical to plaintext evaluation.
- **Transcendental functions** (exponential, trigonometric, sigmoid, softmax): computed directly using standard floating-point arithmetic on the reconstructed masked value $x + r$. The only numerical difference from plaintext evaluation is the inherent rounding error; **no additional approximation** (polynomial, LUT, or otherwise) is introduced by the MPC protocol.
- **Communication rounds**: low constant (1–6)/(1-3) independent of function complexity or number of participants.

6.5 Summary

Table 1 compares representative schemes across key dimensions.

Table 1. Comparison of non-linear function evaluation approaches in secure Transformer inference.

Scheme	Technique	Exact?	Online Rounds	Additional Error
BOLT, BumbleBee, BLB	Polynomial approx.	No	$O(1)$–$O(\log n)$	Polynomial fitting
SecureTLM, SecureGPT	M2A conversion	Yes (floating)	$\lceil \log_2 N \rceil$	None
SIGMA, Mosformer	FSS + LUT	No	Constant	Quantization
Ours	Offline tuples + local eval	Yes (floating)	**1–6**/1-3	**Only random truncation**

7 Conclusion and Future Work

This work presents the first unified framework that enables constant-round, linear-communication MPC for the set of common non-linear functions required by modern machine learning and statistical analysis, with precision guarantees that match plaintext computation—exact for algebraic functions and bitwise equivalent for transcendental functions.

Existing practical MPC solutions for machine learning have been limited to DNNs that use comparison-based activation functions. By providing the first efficient protocols for the full spectrum of non-linear functions used in modern DNNs—with constant rounds, linear communication, and no approximation error—this work removes the final barrier that has prevented MPC from being a truly practical tool for the broader landscape of privacy-preserving machine learning.

Future Work. The efficient offline generation of transcendental correlations $(r, f(r))$ remains an important open problem. While existing techniques (FSS for trigonometric functions, trusted dealer, homomorphic encryption) provide feasible solutions, developing lightweight, PCG-like methods for these non-linear correlations is a promising direction. Additionally, extending our framework to the malicious security model and implementing the protocols for real-world deployment are natural next steps.

References

1. Donald Beaver. Efficient multiparty protocols using circuit randomization. In Joan Feigenbaum, editor, *Advances in Cryptology — CRYPTO '91*, pages 420–432, Berlin, Heidelberg, 1992. Springer Berlin Heidelberg.
2. Donald Beaver. Commodity-based cryptography (extended abstract). In *Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing*, STOC '97, page 446–455, 1997.
3. Elette Boyle, Nishanth Chandran, Niv Gilboa, Divya Gupta, Yuval Ishai, Nishant Kumar, and Mayank Rathee. Function secret sharing for mixed-mode and fixed-point secure computation. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021*, pages 871–900, Cham, 2021. Springer International Publishing.
4. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent ot extension and

more. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019*, pages 489–518, Cham, 2019. Springer International Publishing.

5. Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology - EUROCRYPT 2015*, pages 337–367, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.
6. Yuntian Chen, Xianjia Meng, Zhiying Shi, Zhiyuan Ning, and Jingzhi Lin. Securetlm: Private inference for transformer-based large model with mpc. *Information Sciences*, 667:120429, 2024.
7. Ke Cheng, Yuheng Xia, Anxiao Song, Jiaxuan Fu, Wenjie Qu, Yulong Shen, and Jiaheng Zhang. Mosformer: Maliciously secure three-party inference framework for large transformers. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, CCS '25, page 4124–4138, New York, NY, USA, 2025. Association for Computing Machinery.
8. Ye Dong, Wen-Jie Lu, Yancheng Zheng, et al. PUMA: Secure inference of LLaMA-7B in five minutes. *Security and Safety*, 4(2025014), 2025.
9. Qi Feng, Debiao He, Zhe Liu, Huaqun Wang, and Kim-Kwang Raymond Choo. Securenlp: A system for multi-party privacy-preserving natural language processing. *IEEE Transactions on Information Forensics and Security*, 15:3709–3721, 2020.
10. Kanav Gupta, Neha Jawalkar, Ananta Mukherjee, Nishanth Chandran, Divya Gupta, Ashish Panwar, and Rahul Sharma. Sigma: Secure gpt inference with function secret sharing. *Proc. Priv. Enhancing Technol.*, 2024:61–79, 2024.
11. Meng Hao, Hongwei Li, Hanxiao Chen, Pengzhi Xing, Guowen Xu, and Tianwei Zhang. Iron: Private inference on transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
12. D. Hendrycks and K. Gimpel. Gaussian error linear units (gelus). arXiv:1606.08415, 2016.
13. Yuval Ishai, Joe Kilian, Kobbi Nissim, and Erez Petrank. Extending oblivious transfers efficiently. In Dan Boneh, editor, *Advances in Cryptology - CRYPTO 2003*, pages 145–161, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.
14. Dacheng Li, Rulin Shao, Hongyi Wang, Han Guo, Eric P. Xing, and Hao Zhang. MPCFormer: Fast, performant and private transformer inference with MPC. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
15. Z. Li, C. Xing, Y. Yao, and C. Yuan. Efficient pseudorandom correlation generators over $\mathbb{Z}/p^k\mathbb{Z}$. In Y. Tauman Kalai and S. F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025*, volume 16003 of *Lecture Notes in Computer Science*, Cham, 2025. Springer.
16. Wen-jie Lu, Zhicong Huang, Zhen Gu, Jingyu Li, Jian Liu, Cheng Hong, Kui Ren, Tao Wei, and Wenguang Chen. Bumblebee: Secure two-party inference framework for large transformers. In *Proceedings of the 2025 Network and Distributed System Security Symposium (NDSS)*, 2025.
17. Zihang Zhou Jinxuan DU Min Yang, Dongcan Guo and Qingshu Meng. Esml: An efficient and scalable mpc scheme for privacy-preserving machine learning without precomputation, manuscript, 2026.
18. Payman Mohassel and Peter Rindal. ABY3: A mixed protocol framework for machine learning. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 35–52, 2018.
19. Payman Mohassel and Yupeng Zhang. SecureML: A system for scalable privacy-preserving machine learning. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 19–38. IEEE, 2017.

20. Qi Pang, Jinhao Zhu, Helen Möllering, Wenting Zheng, and Thomas Schneider. Bolt: Privacy-preserving, accurate and efficient inference for transformers. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 4753–4771, 2024.
21. Deevashwer Rathee, Mayank Rathee, Rahul Kranti Kiran Goli, Divya Gupta, Rahul Sharma, Nishanth Chandran, and Aseem Rastogi. Sirnn: A math library for secure rnn inference. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1–18. IEEE, 2021.
22. S. Wagh, D. Gupta, and N. Chandran. SecureNN: 3-party secure computation for neural network training. In *Proceedings on Privacy Enhancing Technologies (PETS) 2019*, 2019.
23. Sameer Wagh. Pika: Secure computation using function secret sharing over rings. *Proceedings on Privacy Enhancing Technologies*, 2022(4):351–377, 2022.
24. Haoqi Wu, Wenjing Fang, Yancheng Zheng, Junming Ma, Jin Tan, Yinggui Wang, and Lei Wang. Ditto: Quantization-aware secure inference of transformers upon MPC. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
25. Pengzhi Xing, Hongwei Li, Meng Hao, Hanxiao Chen, and Shengke Zeng. Trifss: Secure trigonometric function evaluation via function secret sharing. In *IEEE International Conference on Communications, ICC 2023, Rome, Italy, May 28 - June 1, 2023*, pages 1585–1590. IEEE, 2023.
26. Tianshi Xu, Wen-jie Lu, Jiangrui Yu, Yi Chen, Chenqi Lin, Runsheng Wang, and Meng Li. Breaking the layer barrier: remodeling private transformer inference with hybrid ckks and mpc. SEC '25, USA, 2025. USENIX Association.
27. C. Zeng, D. He, Q. Feng, X. Yang, and Q. Luo. SecureGPT: A framework for multi-party privacy-preserving transformer inference in GPT. *IEEE Transactions on Information Forensics and Security*, 19:9480–9493, 2024.
28. Li Zhengyi, Yang Kang, Tan Jin, et al. Nimbus: Secure and efficient two-party inference for transformers. In *Advances in Neural Information Processing Systems*, volume 37, pages 21572–21600, 2024.
29. Lijing Zhou, Qingrui Song, Su Zhang, Ziyu Wang, Xiang-Yu Wang, and Yong Li. Bicoptor 2.0: Addressing challenges in probabilistic truncation for enhanced privacy-preserving machine learning. *ArXiv*, abs/2309.04909, 2024.

Appendix A: Sign Protocol

The sign protocol[17] determines whether the input x is greater than 0, returning 1 if $x > 0$ and 0 otherwise. We lists it here for reader's convenice.

Input: $x = \sum_{i=1}^{n} x_i \mod 2^l$, where each party P_i holds x_i. Let l_x be the effective bit-length of x, 2^l is the modulus of the ring.

Output: Each party obtains $\text{sign}(x)$.

Offline Phase:

- Generate additive secret shares $\langle m_i \rangle$ for random numbers m_i $(i = 0, 1, \ldots, l_x)$.
- Precompute s_i $(i = 1, \ldots, n)$ such that $s = \sum_{i=1}^{n} s_i \mod 2^l$.

- Precompute shares of the i-truncation of s:

$$\langle w_i \rangle = \big\langle \mathrm{Trc}(-s,\, i,\, l - l_x - i) \big\rangle \mod 2^{l_x},$$

  where Trc is the truncation function defined in Bitcoptor 2.0[**?**].
- Precompute shares $\langle M \rangle$ of a random permutation matrix M.
- Send $\langle m_i \rangle$, s_i,$\langle w_i \rangle$ and $\langle M \rangle$ to each party P_i, for $i = 1, 2, \cdots, n$.

Online Phase:

1. Each party P_i broadcasts $x_i + s_i$. After collecting all broadcasts, each party computes

$$x + s = \sum_{j=1}^{n} (x_j + s_j) \mod 2^l.$$

2. Each party locally computes

$$\mathrm{Trc}(x + s,\, j,\, l - l_x - j) \mod 2^{l-j},$$

   and then all parties collaboratively generate its additive secret shares:

$$\langle u_j \rangle = \big\langle \mathrm{Trc}(x + s,\, j,\, l - l_x - j) \big\rangle \mod 2^{l_x}.$$

   Then set

$$\langle u_j \rangle \leftarrow \langle u_j \rangle + \langle w_j \rangle \mod 2^{l_x},$$

   and let

$$\langle v_j \rangle = \langle u_j + u_{j+1} - 1 \rangle \mod 2^{l_x} for j = 0, 1, \cdots, l_x - 1,$$

   and finally

$$\langle v_{l_x} \rangle = \langle u_{l_x} - 1 \rangle \mod 2^{l_x}.$$

3. Switch the modulus of $\langle v_j \rangle$ from $\mathbb{Z}_{2^{l_x}}$ to $\mathbb{Z}_p$ (still denoted $\langle v_j \rangle$), for $j = 0, \ldots, l_x$ in accroding to the rule in Bicoptor 2.0[29].
4. Using the shares $\langle v_j \rangle$ and $\langle m_j \rangle$, each party runs a multiplication protocol to obtain

$$\langle t_j \rangle = \langle v_j \cdot m_j \rangle, \quad j = 0, \ldots, l_x,$$

   which hides each nonzero v_j completely. For $v_j = 0$, it is the target each party wants to find.
5. The parties invoke a multi-party matrix multiplication to compute $\mathsf{Mult}(\langle M \rangle, \langle \{t_0, t_1, \cdots, t_{l_x}\} \rangle)$, obtaining randomly permuted shares $\langle t'_j \rangle$ for $j = 0, \ldots, l_x$. Then each party broadcasts $\langle t'_j \rangle$.
6. Each party computes

$$t'_j = \sum \langle t'_j \rangle \mod p, \quad j = 0, \ldots, l_x.$$

   If there exists a j such that $t'_j = 0$, then $\mathrm{sign}(x) = 1$; otherwise $\mathrm{sign}(x) = 0$.

This protocol require 4 rounds of communication.

Appendix B: GeLU protocol

Definition: GeLU$(x) = x \cdot \Phi(x)$, where Φ is the standard Gaussian cumulative distribution function. There are two common approximation for Gelu(x) in practice [12]: GeLU$(x) \approx x{\cdot}sigmoid(1.702x)$ or GeLU$(x) \approx 0.5x\left(1+\tanh\left(\sqrt{\frac{2}{\pi}}(x+0.044715x^3)\right)\right)$. If we adopt GeLU(x) $= \frac{x}{1+e^{-1.702x}}$, the GeLU protocol is as follows:

Offline Phase: Output the ASS shares $\langle e^s \rangle$ of e^s, with s a random number.

Online pahse:

Input: Each P_i holds $\langle x \rangle_i$ such that $x = \sum_i \langle x \rangle_i$.

Output: Each participant P_i obtains an ASS share of GeLU(x).

Protocol Steps:

1. Run the exponential protocol to obtain shares $\langle e^{-1.702x} \rangle_i$.
2. Each participant takes $\langle e^{-1.702x}+1 \rangle_i$ and $\langle e^s \rangle_i$ as inputs, runs the multiplication protocol to obtain $\langle (e^s e^{-1.702x}+e^s) \rangle_i$, and broadcasts it; each party takes $\langle x \rangle_i$ and $\langle e^s \rangle_i$ as inputs, runs the multiplication protocol to obtain $\langle x(e^s) \rangle_i$.
3. Each participant P_i aggregates the received shares to obtain $t = e^{-1.702x+s} + e^s$, and locally computes $\langle xe^s \rangle_i / t$ as the share of GeLU(x).

The protocol requires 6 rounds of communications and offers provable security. However, we should notice that this protocol is not approximation-free because it is based on an approximated representation.

For GeLU$(x) \approx 0.5x\left(1+\tanh\left(\sqrt{\frac{2}{\pi}}(x+0.044715x^3)\right)\right) = x{\cdot}sigmoid\left(\sqrt{\frac{8}{\pi}}(x+0.044715x^3)\right)$, an extra run of power protocol to comute x^3 is required. That is, the protocol will require 7 rounds of communications.